Artist Revenue Management System

Database Design & Implementation

MDL018 – Data Organization and Management

Member A *ID_A*
Member B *ID_B*
Member C *ID_C*
Member D *ID_D*

Faculty of Information Technology
University of Science – VNU-HCM

February 13, 2026

Table of Contents

Database Objectives

Artist Revenue Management System

Database Design Project

Problem Statement:

- Music artists have revenue from **multiple sources**: streaming, download, live shows
- Need to manage **contracts**, revenue splitting between multiple parties
- Track tracks, albums, bookings and

Database Goals:

- Manage artist info, record labels, albums, tracks
- Record revenue from multiple sources
- Manage contracts & profit distribution
- Statistics, revenue trend analysis

User Personas

Artists

- View personal revenue
- Track tracks, albums
- Check wallet balance
- View booking schedule

Record Labels

- Manage signed artists
- Create & manage contracts
- Distribute revenue
- Monitor revenue logs

Administrators

- System-wide statistics
- Manage venues, bookings
- Revenue reports
- Dashboard analytics

Tech Stack

Layer	Technology	Role
Database	PostgreSQL 16 + pgvector	Storage & queries
Backend	SQL Stored Procedures	Data exploitation
Frontend	Streamlit + Plotly	Dashboard analytics
Infra	Docker Compose	Deployment
VCS	Git + GitHub	Source code management

ERD Overview

13 entities divided into 3 functional groups:

Core Entities

- `artists` (cha)
- `solo_artists` (con)
- `bands` (con)
- `labels`
- `albums`
- `tracks`

Revenue & Contracts

- `revenue_logs`
- `contracts`
- `contract_splits`
- `artist_wallets`

Booking & Venues

- `bookings`
- `venues`
- `managers`

ISA Relationship (parent–child): `artists` → {`solo_artists`, `bands`} — disjoint, total participation

ERD Diagram

(Insert ERD diagram here)
Created with dbdiagram.io or draw.io, export as PNG

ISA Relationship: artists → solo_artists / bands

Why ISA?

- Solo artists and bands have **distinct attributes**
- Solo: main instrument, solo style
- Band: member count, formation date
- Shared: stage_name, debut_date, label_id

Design

`artists` (parent):
PK: artist_id
stage_name, full_name, debut_date
label_id → FK labels

`solo_artists` (child):
PK/FK: artist_id → artists
instrument, solo_style

`bands` (child):
PK/FK: artist_id → artists
member_count, formation_date

Main Relationships

Relationship	Cardinality	Description
artists → solo/bands	ISA (d, t)	Parent–child, disjoint, total
labels → artists	1:N	Label manages multiple artists
artists → albums	1:N	Artist releases multiple albums
albums → tracks	1:N	Album contains multiple tracks
tracks ← revenue_logs	1:N	Track has multiple revenue logs
contracts ↔ artists	M:N (via splits)	Contract splits across parties
artists → bookings	1:N	Artist has multiple bookings
venues → bookings	1:N	Venue hosts multiple shows
managers → bookings	1:N	Manager handles multiple bookings

Physical Design – Main Tables

Attribute	Description	Type	Constraints
Table: artists			
artist_id	Artist ID	SERIAL	PK
stage_name	Stage name	VARCHAR(100)	NOT NULL
full_name	Real name	VARCHAR(150)	NOT NULL
debut_date	Debut date	DATE	
metadata	Social media links	JSONB	
label_id	Label ID	INT	FK $\rightarrow$ labels
Table: tracks			
track_id	Track ID	SERIAL	PK
title	Track title	VARCHAR(200)	NOT NULL
duration	Duration (seconds)	INT	CHECK > 0
album_id	Album ID	INT	FK $\rightarrow$ albums
play_count	Play count	BIGINT	DEFAULT 0
isrc	International code	VARCHAR(12)	UNIQUE

PostgreSQL Implementation

Infrastructure

- **Docker Compose**: PostgreSQL 16 + pgvector + pgAdmin
- SQL scripts run **idempotent** (DROP IF EXISTS)
- Migrations in order: 001_ → 002_ → 003_

Sample Data

Table	Records
labels	5
artists	10
albums	15
tracks	40
revenue_logs	100+
contracts	10
contract_splits	20
bookings	15

Views & Materialized Views

Views (real-time computed)

`v_top_artists`:
Top artists by total plays

`v_top_artists2`:
Top artists by actual revenue

Materialized View (cached)

`mv_top_artist_cached`:
Pre-computed ranking
Faster query for Dashboard
Periodic refresh:

```
REFRESH MATERIALIZED VIEW
mv_top_artist_cached;
```

Overview of 5+ Stored Procedures

#	Stored Procedure	Technique
1	Revenue by artist & month	**ROLLUP**
2	Pivot revenue by source	**crosstab (PIVOT)**
3	Top revenue artists	**Subquery in HAVING**
4	Contract revenue distribution	Subquery + JOIN
5	Booking stats by venue	Window function

Meets project requirements: **nested queries** + **ROLLUP** + **PIVOT**

SP1: Revenue by artist & month (ROLLUP)

```sql
SELECT
    COALESCE(a.stage_name, '*** TOTAL ***') AS artist,
    COALESCE(TO_CHAR(r.log_date, 'MM/YYYY'), 'All') AS month,
    SUM(r.amount) AS total_revenue
FROM revenue_logs r
JOIN tracks t ON r.track_id = t.track_id
JOIN albums al ON t.album_id = al.album_id
JOIN artists a ON al.artist_id = a.artist_id
GROUP BY ROLLUP(a.stage_name, TO_CHAR(r.log_date, 'MM/YYYY'))
ORDER BY a.stage_name NULLS LAST;
```

(Insert screenshot of results here)

SP2: PIVOT revenue by source

```sql
SELECT * FROM crosstab(
    'SELECT a.stage_name, r.source, SUM(r.amount)::NUMERIC
      FROM revenue_logs r
      JOIN tracks t ON r.track_id = t.track_id
      JOIN albums al ON t.album_id = al.album_id
      JOIN artists a ON al.artist_id = a.artist_id
      GROUP BY a.stage_name, r.source
      ORDER BY a.stage_name, r.source',
    'SELECT unnest(ARRAY[''download'',''live'',''streaming''])'
) AS ct(artist VARCHAR, download NUMERIC, live NUMERIC, streaming NUMERIC);
```

(Insert screenshot of results here)

SP3: Top revenue artists (Subquery)

```sql
SELECT a.stage_name, SUM(r.amount) AS total
FROM artists a
JOIN albums al ON a.artist_id = al.artist_id
JOIN tracks t ON al.album_id = t.album_id
JOIN revenue_logs r ON t.track_id = r.track_id
GROUP BY a.artist_id, a.stage_name
HAVING SUM(r.amount) = (
    SELECT MAX(sub.total) FROM (
        SELECT SUM(r2.amount) AS total
        FROM revenue_logs r2
        JOIN tracks t2 ON r2.track_id = t2.track_id
        JOIN albums a2 ON t2.album_id = a2.album_id
        GROUP BY a2.artist_id
    ) sub
);
```

Nested subquery in `HAVING` clause

SP4 & SP5: Contract Splits & Booking Stats

SP4: Revenue Distribution

- JOIN `contracts` + `contract_splits` + `revenue_logs`
- Calculate actual revenue = total × share_percentage
- Compare with `artist_wallets` balance

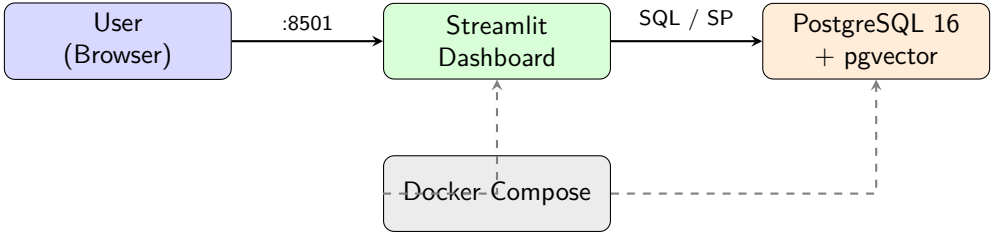
(Screenshot of results)

SP5: Booking by venue

- Stats on shows/venue
- `RANK()` popular venue ranking
- Combined with manager info

(Screenshot of results)

Demo System Architecture



Run everything with 1 command: `docker compose up -d`

Dashboard – Overview Statistics

(Insert Dashboard screenshot here)

Artists	Tracks	Revenue	Bookings
10	40+	500M+	15

Revenue Charts & PIVOT

Revenue by month (ROLLUP)

(Screenshot of Plotly chart)

PIVOT: revenue source × artist

(Screenshot of crosstab table)

Data directly from **Stored Procedures** → Plotly charts

Summary

Completed

- 13 tables with complete PK, FK, CHECK, UNIQUE
- ISA relationship: `artists` → `solo_artists` / `bands`
- Normalized `bookings` from bad design
- 5+ Stored Procedures: ROLLUP, PIVOT, nested subquery
- Views + Materialized View
- Streamlit Dashboard + Docker

Highlights

- **Realistic** sample data (VN/international artists)
- `revenue_logs.raw_data` uses **JSONB** – PostgreSQL advanced
- Materialized View for **query optimization**
- Docker Compose – **1 command** runs everything
- Visual demo with Plotly charts

Team Contributions

No.	Name	Assignment	Contribution
1	Member A	Specification, ERD, design, report	25%
2	Member B	DDL scripts, seed data, views	25%
3	Member C	Stored procedures, exploitation	25%
4	Member D	Streamlit demo, slides, report	25%

Git: 4 members $\times$ feature branches $\rightarrow$ develop $\rightarrow$ main

Commit convention: `feat(db): create core tables`

Thank you for your attention!

Q & A

We are ready to answer your questions.

Source Code & Database:

`https://github.com/<org>/artist-revenue-management`